FANTASY FOOTBALL INTELLIGENCE HUB

Software Design Specification

Architecture, Data Dictionary, Automation, Testing, Deployment, and Revision History

Document control	Current value
Document version	2.3.0
Application baseline	Fantasy Football Intelligence Hub v2.3.0 / APP_VERSION 2.3.0
Status	Living design specification — Version 2.3.0 release
Created	July 26, 2026
Last updated	August 1, 2026
Created by	Mohamad Mokbel
Deployment	GitHub Pages from main branch / repository root
Source of truth	GitHub repository

Version-control decision: Version 2.3.0 is the application baseline. This release adds stateless Sleeper acquisition, on-demand linked-season evidence, Settings-based methodology and weighting, privacy safeguards, and the refined report navigation.
2.2.1

Table of Contents

5.1 Version 2.3 League Analysis Experience

Analyze retrieves the current league data required for the Dashboard, then builds a metadata-only index of linked seasons. The landing page accepts a public Sleeper league ID and Contender Index weights; report persistence, report controls, and advanced detection overrides are removed. Deep transactions, drafts, matchups, and brackets load only when their related view is opened. Progress reflects real work without manufactured delay, and incomplete upstream results remain visible as warnings.

5.2 Dashboard and Team Insights Hierarchy

The Dashboard is now league-centric. It presents the League Scoring History chart and full-width League Power Rankings, keeping its purpose focused on comparing every roster. Focused-team Current Season, Overall Performance, Championship Outlook, and roster-review content moved to Team Insights so users do not see the same analysis twice.

5.3 Focused Team Insights

Team Insights is organized around the selected team: championship recommendation, key current-season and overall-performance context, roster profile, strengths and weaknesses, position/bench/draft reviews, actionable Build and Shop candidates, optimal-lineup evidence, and the full roster. It consumes the same detailed recommendation and position-review data as the analytical model.

League Scoring History renders one concurrent line per roster across linked seasons. Every reported matchup score is a weekly point; the focused team is bold and other teams are faint comparison lines. The alternate view shows one completed-season wins point per season. Previous franchises are matched by owner before roster ID.

All matchup-derived team-performance data is capped at the Week 17 fantasy championship. NFL Week 18 is excluded during matchup fetch, analysis, production aggregation, historical series construction, and chart rendering. A reported zero remains a valid score, while a missing score is not displayed as zero.

5.4 Player Trends Placeholder

Player Trends is intentionally empty while its future feature is under design. Existing optimal-lineup analysis remains on Team Insights alongside its full-roster evidence, avoiding duplicate reports.

5.5 Version 2.3 Header and Navigation

The application uses one sticky responsive header with a larger product logo, release/data/version detail below the brand, Focus Team selection, and league context. Its eight tabs are Dashboard, Team Insights, Trade Center, Draft Capital, Player Trends, Player Tiers, Player Master, and Settings. Overflow menus, creator text, Sleeper identifiers, exports, and report-management controls are absent. Page changes update the URL hash and browser history while preserving the selected focus team.

1. Executive Summary

Fantasy Football Intelligence Hub is a static, browser-based dynasty football analysis application published through GitHub Pages. Version 2.3.0 analyzes public Sleeper leagues, indexes linked seasons without persisting league data, joins projection and dynasty-value sources, builds legal optimal lineups, ranks teams, evaluates risk and draft capital, and presents league-wide and focused-team reports.

Version 2.3.0 adds stateless, on-demand Sleeper history acquisition with bounded concurrency, retries, timeouts, progress messages, session-only caching, and visible partial-failure handling. It moves Methodology into Settings, removes report persistence and exports, protects submitted identifiers, and preserves the strict Week 17 cutoff for fantasy-team performance data.

Current processing boundary: League-specific analysis executes in the user's browser. Linked-season metadata is indexed during Analyze; deeper history is requested only when its related page opens and remains only in session memory. GitHub Actions preprocesses shared public source snapshots, while live APIs remain available as fallbacks when snapshots are empty, stale, or unavailable.

2. Scope and Product Objectives

2.1 In Scope

- Analyze any supported public Sleeper NFL league by league ID.
- Detect format, scoring, roster slots, linked seasons, and future pick horizon.
- Aggregate matchup production and build normalized player records.
- Combine RosterAudit and DynastyProcess market values with RosterAudit and Sleeper projections.
- Calculate legal lineups, depth, risk, contender score, franchise score, player tiers, and draft capital.
- Publish a responsive static application through GitHub Pages.
- Use same-origin snapshots for shared datasets and live fallbacks for resilience.
- Run repeatable Node tests, syntax validation, JSON validation, and scheduled snapshot refresh jobs.

2.2 Out of Scope

- Authenticated private Sleeper leagues.
- Automated KeepTradeCut scraping.
- Application-owned user accounts or centralized private databases.
- Guaranteed availability of third-party APIs.
- Full transaction-based trade impact and waiver reporting.
- Direct nflverse production integration in the current codebase.

3. Current System Context and Architecture

Figure 1 shows the Version 2.x static architecture after GitHub publication and modularization.

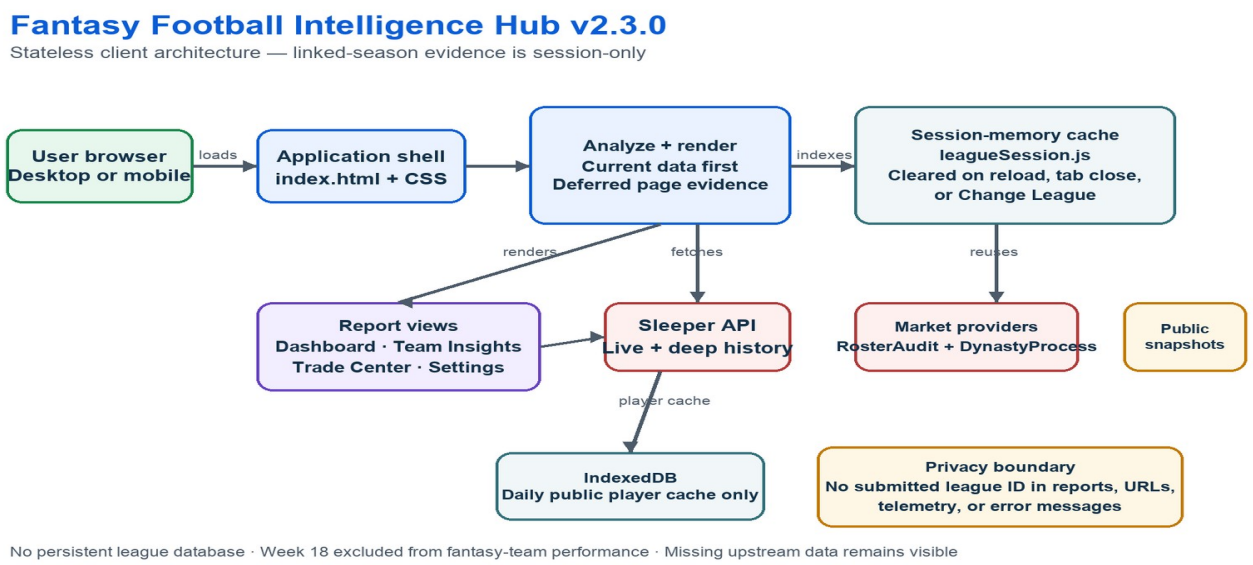

Figure 1. Fantasy Football Intelligence Hub Version 2.3.0 stateless client architecture and data flow.

3.1 Architectural Layers

Layer	Current implementation	Primary responsibility
Hosting	GitHub Pages	Serve index.html, CSS, modules, images, and snapshots over HTTPS.
Application shell	index.html + css/*	Provide branded single-page structure, sticky global header, grouped navigation, and responsive styles.
Startup	js/app.js	Register header events, page history, sorting, session cleanup, startup, and global error listeners.
Orchestration	js/analyze.js + js/render.js	Coordinate current data processing, deferred linked-season evidence, and page rendering.
Domain analytics	js/league.js + js/tiers.js	Calculate lineups, scores, Week 1–17 history, tiers, risk, picks, and insights.
Provider layer	js/api/*	Load snapshots and live external data through source-specific modules.
Views	js/views/*	Render each application page.
Persistence	Session memory + data/*.json	Keep league evidence transient; retain IndexedDB only for the daily public player cache and repository-managed snapshots.
Automation	scripts/* + GitHub Actions	Refresh snapshots, test, validate, and commit data changes.

3.2 End-to-End Runtime Flow

1. User loads the GitHub Pages URL over HTTPS.
2. index.html loads js/app.js as an ES module.
3. The user enters a public Sleeper league ID, optionally adjusts Contender Index weights, and starts analysis through one primary Analyze League action.
4. analyze.js retrieves current Sleeper and shared provider data, detects league setup, builds a metadata-only linked-season index, and creates the initial Dashboard-ready report.
5. Provider modules prefer usable same-origin snapshots and use live public endpoints as fallbacks.
6. league.js and tiers.js build normalized player, team, lineup, pick, score, risk, tier, insight, standing, franchise-summary, and Week 1–17 historical-scoring objects.
7. render.js calls page-specific modules under js/views/.
8. Deferred league evidence is cached only in module-scoped session memory and is cleared on reload, tab close, or Change League. The shared header synchronizes report-page selection with browser history while preserving the selected focus team.

4. Repository and Module Architecture

4.1 Repository Structure

```
fantasy-football-intelligence-hub/
├── index.html
├── assets/images/logo.png
├── css/
├── js/
│   ├── app.js, analyze.js, render.js
│   ├── config.js, state.js, storage.js, utils.js, errors.js
│   ├── league.js, tiers.js
│   ├── api/
│   ├── views/
│   └── tests/
├── data/
├── scripts/
├── docs/
├── .github/workflows/update-data.yml
├── package.json
└── package-lock.json
```

4.2 Runtime Module Responsibilities

Module	Purpose	Used by
js/app.js	Browser entry point. Registers grouped page navigation, Focus Team changes, overflow actions, exports, table sorting, browser history, global errors, and embedded-report startup.	index.html via <script type="module">
js/analyze.js	End-to-end league analysis orchestrator.	End-to-end league analysis, including linked-season matchup history and weekly chart data.
js/api.js	Compatibility export bridge for the provider API directory.	Legacy or future imports expecting js/api.js
js/config.js	Central application version and external endpoint configuration.	analyze.js, api provider modules, update design references
js/errors.js	Central error normalization, reporting, and user-facing message generation.	app.js, analyze.js
js/league.js	Core league rules and analytical calculations.	Core league calculations, historical weekly scoring, and Week 17 production cutoff.
js/render.js	Top-level presentation and application-shell orchestrator, including header context and persistent active-view behavior.	app.js, analyze.js
js/state.js	Single shared in-memory state object.	app.js, render.js, view modules, analyze.js
js/storage.js	IndexedDB cache abstraction.	app.js, analyze.js, api/sleeper.js
js/tiers.js	Player tier scoring and evidence-backed team insight generation.	analyze.js, render.js, views/tiers.js, tests
js/utils.js	Shared DOM, numeric, formatting, CSV, download, and status helpers.	Most runtime modules
js/api/http.js	Low-level HTTP and same-origin snapshot utilities.	All provider modules
js/api/sleeper.js	Sleeper-specific league and player retrieval.	Sleeper league, player, current and linked-season matchup retrieval; Week 18 is excluded.
js/api/rosterAudit.js	RosterAudit value, projection, and pick provider.	analyze.js
js/api/dynastyProcess.js	DynastyProcess identity and market-value provider.	analyze.js
js/api/projections.js	Sleeper projection response normalization and fallback loading.	analyze.js
js/api/index.js	Public barrel export for provider modules.	analyze.js, api.js bridge
js/views/shared.js	Shared presentation helpers for tables and evidence blocks.	Dashboard, team, lineup, trade, and player views
js/views/dashboard.js	League dashboard page renderer.	League-centric dashboard with scoring history and power rankings.
js/views/teams.js	Team Insights page renderer.	Focused-team recommendations, reviews, candidates, lineup evidence, and full roster.
js/views/lineups.js	Optimal Lineups page renderer.	Intentionally blank Coming Soon! placeholder.
js/views/trade.js	Trade Finder page and target-ranking helpers.	render.js
js/views/players.js	Player Master page renderer.	render.js
js/views/tiers.js	Player Tiers page renderer.	render.js
js/views/picks.js	Draft Capital page renderer.	render.js
js/views/methodology.js	Methodology page renderer.	render.js

4.3 Companion JavaScript Documentation

Every JavaScript and Node automation file has a companion `<filename>.README.md` file located beside the source file. The master index is stored at `docs/JAVASCRIPT_FILE_DOCUMENTATION_INDEX.md`. These files explain purpose, responsibilities, dependencies, consumers, and maintenance constraints.

5. Application Components and User Experience

Component	Responsibility
Landing / League Selection	One primary Analyze League action; automatic format, linked-season, and future-pick detection; Contender Ranking Weights; advanced overrides; paced loading; and secondary previous-report/live-data actions.
Persistent Header	Sticky logo-based application header with grouped report-page selector, league name, detected settings, analysis timestamp, Focus Team selector, data status, version badge, responsive behavior, browser-history synchronization, and overflow actions.
Dashboard	League Scoring History (all reported Week 1–17 scores, focused team bold, comparison lines faint, completed-season wins view) and full-width League Power Rankings.
Team Insights	Focused-team championship recommendation, current and historical context, roster profile, strengths/weaknesses, position/bench/draft reviews, Build/Shop candidates, optimal-lineup evidence, and full roster.
Optimal Lineups	Coming Soon! placeholder; optimal-lineup evidence is retained on Team Insights.
Trade Finder	Projected upgrades to the weakest legal slot, ranked by gain relative to cost.
Player Master	Filterable rostered and relevant free-agent analytical records.
Player Tiers	Tier S and Tiers 1-4 using projection, value, and longevity.
Draft Capital	Capital leaderboard, portfolios, ownership matrix, and movement.
Methodology	Formulas, weights, sources, limitations, and interpretation.
Exports	Analysis JSON, ranking CSV, player CSV, embedded HTML, and print/PDF.

6. Data Processing and Analytical Logic

6.1 Identity and Normalization

- Sleeper player ID remains the primary runtime key.
- DynastyProcess supplies FantasyPros, GSIS, KeepTradeCut, draft, college, and secondary value fields when available.
- Normalized names are fallback joins only.
- Free agents are included when active or when projection/value signals exceed relevance thresholds.
- Expected PPG remains projected season total divided by 17.

6.2 Core Formulas

Metric	Rule
Optimal lineup	Minimum-cost flow with negative expected PPG as edge cost; each player used at most once; taxi excluded.
Depth	45% of the sum of expected PPG for the top six eligible bench players.
Consensus dynasty value	Mean of available RosterAudit and DynastyProcess values.
Contender score	40% projection + 15% depth + 15% production + 15% dynasty + 10% low risk + 5% picks.
Franchise score	40% dynasty + 25% young value + 15% picks + 10% projection + 10% low risk.
Player tier composite	60% same-position expected-PPG percentile + 25% value percentile + 15% longevity percentile.
Risk	Rule-based 0-100 score using age, injury/status, NFL team, depth order, projected games, and value fragility.

7. External Data Sources and Provider Design

Provider	Module	Snapshot	Live fallback	Primary data
Sleeper league	api/sleeper.js	Player directory only	Yes	League, users, rosters, matchups through Week 17, picks, drafts, NFL state.
Sleeper projections	api/projections.js	data/projections.json	Yes	Projected fantasy totals and games.
RosterAudit	api/rosterAudit.js	Values, projections, picks JSON	Yes	Primary dynasty values, trends, projections, pick information.
DynastyProcess	api/dynastyProcess.js	data/dynasty-process.json	Yes	Secondary values and cross-platform identifiers.

7.1 Provider Fallback Pattern

9. Attempt to load the same-origin snapshot.
10. Validate that the snapshot has usable content and, where relevant, the correct season or format.
11. Use the live public endpoint when the snapshot is empty or unsuitable.
12. Return a source label so status badges and diagnostics distinguish snapshot from live data.
13. Use Promise.allSettled for optional providers so Sleeper league analysis can continue when a market source fails.

8. Snapshot Automation and GitHub Actions

8.1 Snapshot Files

File	Top-level fields	Purpose
data/players.json	generatedAt, players	Sleeper player directory keyed by player ID.
data/projections.json	generatedAt, season, players	Season-specific Sleeper projections.
data/dynasty-values.json	generatedAt, formats	RosterAudit values grouped by formatKey.
data/roster-audit-projections.json	generatedAt, data	RosterAudit projection payload.
data/pick-values.json	generatedAt, data	RosterAudit pick payload.
data/dynasty-process.json	generatedAt, ids, values	Parsed DynastyProcess ID and value rows.
data/metadata.json	generatedAt, season, nflState, sources	Automation run lineage and source outcomes.

8.2 Scheduled Workflow

`.github/workflows/update-data.yml` supports manual execution and a daily UTC cron schedule.

14. Check out the main repository.
15. Set up Node.js 22 with npm cache.
16. Run npm ci.
17. Run the calculation tests.
18. Run scripts/update-data.mjs.
19. Run project validation.
20. Commit changed data files with the GitHub Actions bot identity.

Failure behavior: The update script preserves the previous snapshot when an optional source fails. The workflow fails only when all source updates fail or a test/validation step fails.

9. Storage, Caching, and Export Design

Mechanism	Key / grain	Purpose	Scope
IndexedDB player cache	players:YYYY-MM-DD	Avoid repeated large Sleeper directory downloads.	Per browser/device.
IndexedDB analysis cache	analysis:{leagueId}	Reopen a completed report without rerunning analysis.	Per browser/device.
Repository snapshots	One JSON file per shared dataset	Improve reliability and same-origin loading.	Shared/public.
Analysis JSON export	One analysis object	Auditable machine-readable output.	User download.
CSV exports	One row per team or player	Spreadsheet analysis.	User download.
Embedded HTML export	HTML plus embedded analysis JSON	Portable offline report.	User download.

10. Testing, Validation, Error Handling, and Reliability

10.1 Automated Tests

Test file	Coverage
js/tests/league.test.js	Eligibility, percentiles, risk, optimal lineup, pick ownership, setup detection, linked-season summaries, weekly team history, and Week 18 production exclusion.
js/tests/tiers.test.js	Tier scoring, sort order, inactive-player exclusion, championship outlook generation, and Build/Shop candidate rules.
js/tests/header.test.js	Global header structure, page navigation, and Version 2.2.1 consistency.
js/tests/dashboard.test.js	League-history series, zero-score retention, completed-season wins, and Week 18 chart exclusion.
js/tests/teams.test.js	Selected-team fallback, section order, candidate fallbacks, safe empty data, lineup columns, and evidence metrics.

10.2 Validation

- `npm test` executes Node's built-in test runner.
- `npm run validate` verifies required files, JavaScript syntax, JSON parsing, and the module script reference.
- `npm run check` runs both tests and validation.
- GitHub Actions runs tests and validation before committing snapshot updates.

10.3 Error Handling

`js/errors.js` defines AppError, normalizes timeout/network/unexpected failures, creates user-facing messages, and logs structured diagnostics. `app.js` registers global error and unhandled-rejection listeners. `analyze.js` reports provider or processing failures without exposing raw stack traces in the normal status UI.

10.4 Reliability Controls

- Optional source isolation with Promise.allSettled.
- Snapshot-first provider design with live fallback.
- Daily player cache and explicit Force Refresh.
- Source badges showing availability and snapshot/live origin.
- Automated project validation and GitHub Actions logs.
- GitHub Pages HTTPS deployment and relative asset paths.

11. Deployment, Security, and Operations

11.1 Deployment

The application is deployed from the GitHub repository main branch and repository root through GitHub Pages. All paths are relative so the application works under the project path `/fantasy-football-intelligence-hub/`. The repository is the authoritative source; local files are development working copies only.

11.2 Security and Privacy

- No credentials are collected for public Sleeper league analysis.
- No secrets are stored in client-side code or required by the current automation.
- All production requests use HTTPS.
- Exports may contain public manager names, team names, league IDs, and roster details and remain user-controlled.
- GitHub Actions uses repository content permissions only for committing snapshot changes.

11.3 Operational Workflow

21. Pull current main.
22. Create a feature branch.
23. Implement or merge generated changes.
24. Run npm check and browser regression tests.
25. Commit and push the feature branch.
26. Merge to main through a pull request.
27. Confirm GitHub Pages deployment and scheduled workflow health.

12. Data Dictionary and Source Lineage

12.1 Analysis Run Object

Field	Type	Source	Definition
appVersion	string	Application constant	Version that generated the analysis.
generatedAt	datetime	Browser timestamp	Analysis creation time.
leagueId	string	Sleeper	Primary league identifier.
leagueName	string	Sleeper	League display name.
season	integer	Sleeper	Fantasy season.
leagueStatus	string	Sleeper	League lifecycle status.
currentWeek	integer	Sleeper NFL state	Highest fetched current-season week; limited to Week 17.
totalRosters	integer	Sleeper rosters	Number of fantasy rosters.
formatKey	string	Derived	Internal source-selection key.
formatLabel	string	Derived	Human-readable format.
rosterSlots	array<string>	Derived	Legal starting slots.
unsupportedSlots	array<string>	Derived	IDP slots with incomplete market support.
weights	object	User configuration	Normalized contender-score weights.
useCurrentProduction	boolean	Derived	Current vs prior production selection.
teams	array<team>	Derived	Team analytical records.
players	array<player>	Derived	Rostered and relevant free-agent records.
tierPlayers	array<player>	Derived at render preparation	Tier-eligible QB/RB/WR/TE records.
picks	array<pick>	Sleeper + derived	Future pick ownership model.
sourceStatuses	array<object>	Runtime	Provider status and origin.
history	array<object>	Sleeper	Linked league seasons.
methodology	object	Application	Human-readable calculation descriptions.
detectedSetup	object	Sleeper league + linked history	Detected team count, format, starter count, IDP presence, history range, and future-pick range.

12.2 Player Analytical Record

Field group	Type	Definition
rosterId	integer	Sleeper roster or 0 for free agent
fantasyTeam	string	Fantasy team or Free Agent
manager	string	Sleeper manager display name
sleeperId	string	Primary player key
name	string	Display name
position	string	QB/RB/WR/TE or other source position
nflTeam	string	NFL team abbreviation
age	number|null	Player age
status / injuryStatus / depthOrder	mixed	Availability and role fields
rosterStatus	string	Starter, Bench, Reserve, Taxi, or Free Agent
actualPoints / actualPpg	decimal	Fetched production aggregate
rosteredWeeks / scoringWeeks / starts	integer	Production sample and usage
starterPpg	decimal	Points in fantasy starts / starts
projectedTotal / projectedGames	decimal|null	Normalized projection
expectedPpg	decimal	projectedTotal / 17
sourcePpg / projectionSource	mixed	Projection lineage
dynastyValueRA / dynastyValueDP	decimal|null	Source-specific values
dynastyValue	integer	Mean of available values
trend7d / trend30d	decimal|null	RosterAudit market trends
draftYear / draftRound / college	mixed	Identity and pedigree
gsisId / fantasyProsId / ktcId	string	Cross-platform identifiers
riskScore / riskTier / reasons	mixed	Rule-based risk model
positionRank / positionPoolSize	integer	League positional context
positionPercentile / positionTier	mixed	Projection-based positional tier
longevityScore / longevityPct	decimal	Tier longevity component
projectionPct / valuePct	decimal	Same-position tier components
tierComposite / analysisTier	mixed	Player Tiers composite and S/1-4 label

12.3 Team Analytical Record

Field group	Definition
rosterId, team, manager	Identity
players, lineup, bench	Roster and legal-lineup collections
positionScores	Projected positional group totals
lineupPpg, lineupSourcePpg	Projected lineup strength
depth	Top-six bench index
risk, lineupAge, highRiskStarters	Projected-lineup risk profile
totalValue, lineupValue, youngValue	Dynasty asset measures
currentWins/Losses/PF/MaxPF	Current production
priorWins/Losses/PF/MaxPF	Historical baseline
pickCapital, futureFirsts, picksOwned	Draft capital
projectionPct, depthPct, productionPct, dynastyPct, riskPct, picksPct, youngPct	League-relative components
contenderScore, currentRank, currentClass	Current competitive ranking
franchiseScore, franchiseRank, franchiseClass	Long-term ranking
insights	Evidence-backed strengths, weaknesses, strategy, Build candidates, Shop candidates, championship outlook, and position reviews.
historical	Linked-season franchise summary: matched seasons, completed seasons, record, points, average PPG, finish, rank, and standing.
positionRanks	League rank for QB, RB, WR, TE, and FLEX projected starter output.
biggestStrength, biggestWeakness	Best and worst positional ranks used in the full-width power-ranking report.
insights.championshipOutlook	Recommendation title, expanded explanation, confidence, and summary metrics. No separate potential-moves list is rendered.
insights.positionReviews	Narrative and structured review objects for QB, RB, WR, TE, FLEX, Bench Depth, and Draft Capital.
weeklyHistory	Chart-ready linked-season weekly matchup scores through the Week 17 championship, matched to the franchise by owner first.

12.4 Snapshot Data Dictionary

Field	Type	File	Definition
generatedAt	datetime|null	All snapshot files	Time the automation generated the file.
season	integer|null	projections.json, metadata.json	NFL season represented.
players	object	players.json	Sleeper player directory keyed by player ID.
players	array<object>	projections.json	Raw normalized projection rows.
formats	object	dynasty-values.json	Map of formatKey to RosterAudit value payload.
data	object	roster-audit-projections.json / pick-values.json	Provider payload preserved for browser normalization.
ids	array<object>	dynasty-process.json	Parsed db_playerids.csv rows.
values	array<object>	dynasty-process.json	Parsed values.csv rows.
nflState	object	metadata.json	Sleeper NFL state at job time.
sources	object	metadata.json	Per-source success or error outcome.

12.5 Source Status Object

Field	Type	Definition
name	string	Human-readable source name.
status	string	ok, warn, or bad.
detail	string	Live/snapshot/cache origin, count, or error summary.

13. Architecture Decision Record - Version 2.0 Refactor

Decision	Rationale	Consequence
Use ES modules	Separate concerns and reduce monolithic change risk.	Requires HTTP hosting; GitHub Pages satisfies this.
Provider-specific API files	Make source behavior and fallbacks explicit.	More files, but clearer ownership and testing boundaries.
View-specific render files	Reduce render.js size and isolate UI changes.	Shared markup must remain in views/shared.js.
Same-origin JSON snapshots	Reduce CORS and availability risk for shared datasets.	Requires scheduled refresh and schema governance.
GitHub Actions automation	Create repeatable public-data refreshes without a backend.	Workflow health and permissions must be monitored.
Node tests and validation	Catch analytical and structural regressions before deployment.	Node 22+ is required for local checks.
GitHub Pages deployment	Provide free HTTPS static hosting and a single	All paths must be relative and case-correct.

Decision	Rationale	Consequence
	published URL.	
Release Version 2.2	The global header is a material user-facing navigation and responsive-shell enhancement.	Application and documentation versions advance to 2.2.0.

14. Revision History

Date	Document version	Application baseline	Change
July 26, 2026	1.0	v1.x	Initial architecture and data-dictionary working draft.
July 27, 2026	2.0	v2.0	Added Player Tiers and consolidated the SDS.
July 27, 2026	2.0	v2.0 / 2.0.0	Documented external CSS and ES-module architecture, provider and view modules, centralized errors, Node tests, local snapshots, GitHub Actions automation, GitHub Pages deployment, per-file JavaScript READMEs, updated data dictionary, and revised architecture diagram. No application-version increment.
July 28, 2026	2.1	v2.1 / 2.1.0	Released landing-page simplification and automatic league detection; added paced loading, Current Season and Overall Performance summaries, historical franchise metrics, full-width League Power Rankings, QB/RB/WR/TE/FLEX ranks, biggest strength and weakness, and the Championship Outlook & Roster Review.
July 28, 2026	2.2	v2.2 / 2.2.0	Replaced the standalone page-selector banner with a sticky responsive global header containing grouped page navigation, league context, Focus Team selection, data status, version, overflow actions, browser-history support, and active-page persistence.
July 29, 2026	2.2.1	v2.2.1 / 2.2.1	Merged Dashboard and Team Insights refinement: made Dashboard league-centric; moved selected-team performance and recommendations to Team Insights; added linked-season League Scoring History, Build/Shop candidates, full-roster evidence, Coming Soon! placeholder, Week 18 exclusion, and renderer tests.
August 1, 2026	2.3.0	v2.3.0 / 2.3.0	Released stateless Sleeper acquisition with session-only deep-history loading, retries and partial-failure visibility; moved Methodology into Settings; removed report persistence and exports; refreshed header and navigation labels; and documented Week 18 exclusion and identifier privacy.

Date	Document version	Application baseline	Change
August 1, 2026	2.3.0	v2.3.0 / 2.3.0	Released stateless Sleeper acquisition with session-only deep-history loading, retries and partial-failure visibility; moved Methodology into Settings; removed report persistence and exports; refreshed header and navigation labels; and documented Week 18 exclusion and identifier privacy.

15. Release Maintenance Checklist

- Confirm APP_VERSION and displayed Version Control are intentionally unchanged or updated.
- Update SDS sections affected by architecture, analytical, provider, data, deployment, or UX changes.
- Update the data dictionary for every added, changed, or deprecated field.
- Update the architecture diagram when processing boundaries or integrations change.
- Update companion JavaScript README files when module responsibilities or dependencies change.
- Run npm test, npm run validate, and npm run check.
- Test all application tabs through the local HTTP server, including desktop and narrow layouts.
- Confirm relative asset paths and case sensitivity.
- Push through a feature branch and merge to main.
- Confirm GitHub Pages and the Update fantasy football data workflow succeed.

Version 2.2.1 Active Tab Style Patch

Version 2.2.1 corrects the active state of the global report tabs without changing navigation behavior or the desktop header structure.

- Active and inactive tabs use the same height, width allocation, padding, margin, border radius, and vertical position.
- The active state changes only the background to the product blue, the text to white, and the font weight to bold.
- Raised positioning, active-state height growth, negative overlap, visual bridge pseudo-elements, and active-state shadows are removed.
- The tab row no longer overlaps or covers the report content below it.
- Tab click navigation, focus-team persistence, URL hash state, and browser Back/Forward behavior remain unchanged.
- Regression tests verify version consistency, fixed dimensions, color-only active styling, and absence of the previous pseudo-element bridge.

Version 2.2.1 Header Starter Count Metadata Correction

- The league header resolves starter count from a valid stored starterCount value, detectedSetup.starterCount, starterSlots, rosterSlots, or Sleeper roster_positions.
- Bench, IR, reserve, and taxi slots are excluded from derived starter counts. Legal offensive, kicker, defense, Superflex, and IDP starter slots are included.
- New analyses retain starterCount and starterSlots alongside rosterSlots in the in-memory report for the current browser session.
- When no valid starter count can be resolved, the header omits the starter-count segment instead of displaying Starter count unavailable.
- For the referenced 12-team 1QB PPR league, the header renders 12 teams · 1QB PPR · Start 10.
- Regression tests cover direct stored values, detected setup fallback, roster-slot arrays, Sleeper roster positions, excluded reserve slots, and unresolved metadata.

Starter Count Derived Field

starterCount resolution order: explicit analysis starterCount; detectedSetup.starterCount; starterSlots; rosterSlots; direct or nested Sleeper roster_positions. The derived output is a positive integer or null.

Version 2.3.0 — Stateless Sleeper Acquisition

Version 2.3.0 makes linked Sleeper league history available without a persistent league database. Analyze loads live current-season data and a linked-season index first, then page-level resources load on demand and are kept only in browser memory for the current tab.

- Settings now contains Contender Index weights, Methodology, and collapsed League History Evidence.
- Team Insights requests linked-season roster and Week 1–17 matchup history only when opened.
- Deep evidence requests use bounded concurrency, retry and timeout behavior, progress updates, and visible partial-data warnings.
- League IDs and Sleeper user identifiers are excluded from report payloads, URLs, header text, and application error messages.
- NFL Week 18 is excluded from fantasy-team performance metrics and charts.
- Report persistence, exports, printing, advanced detection overrides, and header overflow controls are removed.
- Report tabs are Dashboard, Team Insights, Trade Center, Draft Capital, Player Trends, Player Tiers, Player Master, and Settings.